

Tema 2

Sa va construit un nou model de robot care sa aiba minim 10 componente, dintre care minim 4 trebuie sa poata sa se miste (orice fel de miscare, la alegere). - Miscare elementelor sa se faca prin conectarea la un topic pe care se publica de pe un nod facut de voi. Informatia publicata trebuie sa permita fiecarui element mobil sa realizeze automat o miscare. Cum arata si cum implementati nodul/topicul e la alegerea fiecaruia dintre voi.

1. Crearea structurii unui Workspace și al unui Pachet

Un workspace este un mediu de dezvoltare structurat, folosit pentru a organiza, construi si testa pachetele ROS2, pentru a face acest lucru, vom folosi terminalul:

```
$ cd
```

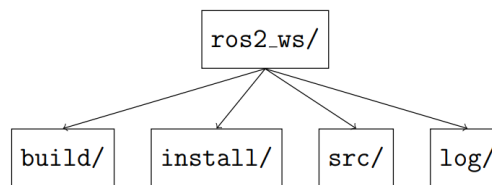
```
$ mkdir ros2_ws
```

Apoi, este necesar să creem un director src, urmând ca în acesta sa creem pachetele ROS2:

```
$ cd /home/thor/ros2_ws
```

```
$ mkdir src
```

Acum, structura workspace-ului arată astfel, aceasta ne va ajuta să gestionam eficient pachetele si procesul de build.



Directoarele de mai sus au următoarele scopuri:

- Ros2.ws - directorul rădăcină al workspace-ului.
- Build - conține fișiere temporare generate în timpul procesului de compilare
- Install - conține fișierele instalate după rularea comenzii **colcon build**
- Src - conține codul sursă al aplicației, aici vor fi plasate toate pachetele ROS2
- Log/ - stochează log-uri ale procesului de build, utile pentru debugging.

Acum că avem workspace-ul creat putem să dezvoltăm structura pachetului nostru, pentru a face asta trebuie sa ne aflam in directorul **src**.

```
$ cd /home/thor/ros2_ws/src
```

Forma generală a comenzii pentru a crea un pachet este de forma:

```
$ ros2 pkg create <nume_pachet> --build-type <tip_build> --dependencies <lista_dependente>
```

Elementele din comanda sunt următoarele:

- ros2 pkg create <nume_pachet> - comanda pentru a crea un pachet
- --build-type <tip_build> - se selectează tipul de sistem de build, pentru C++ se folosește ament_cmake
- --dependencies <lista_dependențe> - adaugă una sau mai multe dependențe necesare pachetului

Acum putem genera automat structura pachetului cu ajutorul comenzii:

```
$ ros2 pkg create --build-type ament_cmake brat_robotic_description
```

Output

```
thor@thor-VM:~$ cd ros2_ws/src/
thor@thor-VM:~/ros2_ws/src$ ros2 pkg create --build-type ament_cmake brat_robotic_description
going to create a new package
package name: brat_robotic_description
destination directory: /home/thor/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['thor <thor@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: []
creating folder ./brat_robotic_description
creating ./brat_robotic_description/package.xml
creating source and include folder
creating folder ./brat_robotic_description/src
creating folder ./brat_robotic_description/include/brat_robotic_description
creating ./brat_robotic_description/CMakeLists.txt

[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the package.xml, but no LICENSE file has been created.
Trash-recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
```

Crearea fișierelor necesare și eliminarea celor inutile:

```
$ cd brat_robotic_description
```

```
$ mkdir urdf launch rviz
```

```
$ rm -rf include/ src/
```

Pentru a vizualiza structura actuală vom folosi comanda ls:

```
$ ls
```

Output

```
thor@thor-VM:~/ros2_ws/src/brat_robotic_description$ ls
CMakeLists.txt  launch  package.xml  rviz  urdf
```

Directoarele de mai sus au următoarele scopuri:

- CMakeLists.txt – fișier ce conține instrucțiuni de compilare și instalare
- Launch – folder pentru fișierele de pornire automata
- Package.xml - fișier în care se regăsesc metadate (nume pachet, versiune, autor , dependențe)
- Rviz – folder pentru configurările de vizualizare
- Urdf – folder pentru modelul robotului

2. Configurarea pachetului și definirea modelului (URDF)

În această etapă, am configurat mediul de dezvoltare și am scris codul care definește structura fizică a robotului. Codul respectă arhitectura bazată pe Publisher-Subscriber.

```
$ cd /home/thor/ros2_ws/src/ brat_robotic_description
```

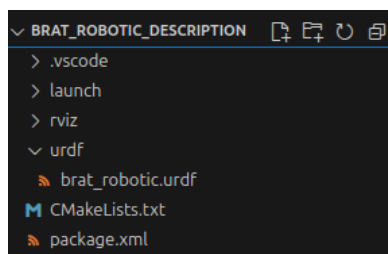
```
$ code .
```

Se deschide Visual Studio Code și editam fișierul CMakeLists.txt:

```
if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  install(
    DIRECTORY launch urdf rviz
    DESTINATION share/${PROJECT_NAME}
  )
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
```

Comanda install pe care am adaugat-o spune sistemului de compilare să copieze folderele launch, urdf și rviz din zona de lucru în zona de instalare a sistemului ROS

Vom crea un document .urdf, dăm click dreapta pe folderul urdf, dăm "New File" și punem numele "brat_robotic.urdf".



Documentul URDF este un fișier XML în care vom descrie cum arata robotul, adică ce piese are și cum sunt conectate.

```
urdf > brat_robotic.urdf
1  <?xml version="1.0"?>
2  <robot name="brat_robotic">
3
4  <!-- culori robot -->
5    <material name="grey">
6      <color rgba="0.5 0.5 0.5 1.0"/>
7    </material>
8
9    <material name="blue">
10     <color rgba="0 0 0.8 1.0"/>
11   </material>
12
13   <material name="red">
14     <color rgba="0.8 0 0 1.0"/>
15   </material>
16
17   <material name="green">
18     <color rgba="0 0.8 0 1.0"/>
19   </material>
20
21   <material name="orange">
22     <color rgba="1.0 0.5 0 1.0"/>
23   </material>
24
25   <!-- baza fixa -->
26   <link name="base_link">
27     <visual>
28       <geometry>
29         <box size="0.1 0.1 0.05"/>
30       </geometry>
31       <origin xyz="0 0 0" rpy="0 0 0"/>
32       <material name="grey"/>
33     </visual>
34   </link>
35
36   <!-- Placa baza fixa -->
37   <link name="base_plate">
38     <visual>
39       <geometry>
40         <cylinder radius="0.15" length="0.05"/>
41       </geometry>
```

```

42     <origin xyz="0 0 0" rpy="0 0 0"/>
43     <material name="grey"/>
44   </visual>
45 </link>
46
47 <!-- conexiune fixa intre base_link si base_plate -->
48 <joint name="base_to_plate" type="fixed">
49   <parent link="base_link"/>
50   <child link="base_plate"/>
51   <origin xyz="0 0 0.05" rpy="0 0 0"/>
52 </joint>
53
54 <!-- brat 1 - inferior -->
55 <link name="column">
56   <visual>
57     <geometry>
58       <cylinder radius="0.04" length="0.2"/>
59     </geometry>
60     <origin xyz="0 0 0.1" rpy="0 0 0"/>
61     <material name="blue"/>
62   </visual>
63 </link>
64
65 <!-- conexiune fixa intre base_plate și column -->
66 <joint name="plate_to_column" type="fixed">
67   <parent link="base_plate"/>
68   <child link="column"/>
69   <origin xyz="0 0 0.025" rpy="0 0 0"/>
70 </joint>
71
72 <!-- articulatie 1 (umar) -->
73 <link name="shoulder_link">
74   <visual>
75     <geometry>
76       <box size="0.08 0.08 0.08"/>
77     </geometry>
78     <origin xyz="0 0 0" rpy="0 0 0"/>
79     <material name="red"/>

```

```

80     </visual>
81 </link>
82
83 <!-- articulatie mobila rotatie pe axa Z - sus jos -->
84 <joint name="base_to_shoulder" type="continuous">
85     <parent link="column"/>
86     <child link="shoulder_link"/>
87     <origin xyz="0 0 0.2" rpy="0 0 0"/>
88     <axis xyz="0 0 1"/>
89     <limit effort="10" velocity="1.0"/>
90     <dynamics damping="0.1" friction="0.1"/>
91 </joint>
92
93 <!-- brat 2 - superior -->
94 <link name="upper_arm">
95     <visual>
96         <geometry>
97             <box size="0.06 0.06 0.3"/>
98         </geometry>
99         <origin xyz="0 0 0.15" rpy="0 0 0"/>
100        <material name="green"/>
101    </visual>
102 </link>
103
104 <!-- conexiune fixa intre shoulder si upper_arm -->
105 <joint name="shoulder_to_upperarm" type="fixed">
106     <parent link="shoulder_link"/>
107     <child link="upper_arm"/>
108     <origin xyz="0 0 0.04" rpy="0 0 0"/>
109 </joint>
110
111 <!-- articulatie 2 (cot) -->
112 <link name="elbow_link">
113     <visual>
114         <geometry>
115             <sphere radius="0.05"/>
116         </geometry>
117         <origin xyz="0 0 0" rpy="0 0 0"/>

```

```

118     <material name="orange"/>
119   </visual>
120 </link>
121
122   <!-- articulatie mobila rotatie pe axa Y - stanga dreapta -->
123   <joint name="shoulder_to_elbow" type="revolute">
124     <parent link="upper_arm"/>
125     <child link="elbow_link"/>
126     <origin xyz="0 0 0.3" rpy="0 0 0"/>
127     <axis xyz="0 1 0"/>
128     <limit lower="-1.57" upper="1.57" effort="10" velocity="1.0"/>
129     <dynamics damping="0.1" friction="0.1"/>
130   </joint>
131
132   <!-- brat 3 - antebrat -->
133   <link name="forearm">
134     <visual>
135       <geometry>
136         <box size="0.05 0.05 0.25"/>
137       </geometry>
138       <origin xyz="0 0 0.125" rpy="0 0 0"/>
139       <material name="blue"/>
140     </visual>
141   </link>
142
143   <!-- conexiune fixa intre elbow si forearm -->
144   <joint name="elbow_to_forearm" type="fixed">
145     <parent link="elbow_link"/>
146     <child link="forearm"/>
147     <origin xyz="0 0 0.05" rpy="0 0 0"/>
148   </joint>
149
150   <!-- articulatie 3 (incheietura) -->
151   <link name="wrist_link">
152     <visual>
153       <geometry>
154         <cylinder radius="0.04" length="0.06"/>
155       </geometry>

```

```

155     </geometry>
156     <origin xyz="0 0 0" rpy="0 0 0"/>
157     <material name="red"/>
158   </visual>
159 </link>
160
161 <!-- articulatie mobila: rotatie pe axa Y - sus jos -->
162 <joint name="elbow_to_wrist" type="revolute">
163   <parent link="forearm"/>
164   <child link="wrist_link"/>
165   <origin xyz="0 0 0.25" rpy="0 0 0"/>
166   <axis xyz="0 1 0"/>
167   <limit lower="-1.57" upper="1.57" effort="10" velocity="1.0"/>
168   <dynamics damping="0.1" friction="0.1"/>
169 </joint>
170
171 <!-- mana -->
172 <link name="hand">
173   <visual>
174     <geometry>
175       <box size="0.06 0.04 0.08"/>
176     </geometry>
177     <origin xyz="0 0 0.04" rpy="0 0 0"/>
178     <material name="grey"/>
179   </visual>
180 </link>
181
182 <!-- conexiune fixa intre wrist si hand -->
183 <joint name="wrist_to_hand" type="fixed">
184   <parent link="wrist_link"/>
185   <child link="hand"/>
186   <origin xyz="0 0 0.03" rpy="0 0 0"/>
187 </joint>
188
189 <!-- articulatie 4 - efector -->
190 <link name="gripper">
191   <visual>
192     <geometry>

```

```

192     <geometry>
193       <box size="0.04 0.06 0.04"/>
194     </geometry>
195     <origin xyz="0 0 0" rpy="0 0 0"/>
196     <material name="orange"/>
197   </visual>
198 </link>
199
200   <!-- articulatie mobila: rotatie pe axa Z -->
201   <joint name="wrist_to_gripper" type="continuous">
202     <parent link="hand"/>
203     <child link="gripper"/>
204     <origin xyz="0 0 0.08" rpy="0 0 0"/>
205     <axis xyz="0 0 1"/>
206     <limit effort="10" velocity="1.0"/>
207     <dynamics damping="0.1" friction="0.1"/>
208   </joint>
209
210 </robot>

```

Următorul pas va fi să compilam pachetul și să îl rulăm. Mai întâi revenim în rădăcina workspace-ului.

```
$ cd /home/thor/ros2_ws
```

Compilarea pachetului se face cu ajutorul comenzii **colcon build**, pentru a nu compila tot workspace-ul, vom selecta doar pachetul creat acum.

```
$ colcon build --packages-select brat_robotic_description
```

```

thor@thor-VM:~/ros2_ws$ colcon build --packages-select brat_robotic_description
Starting >>> brat_robotic_description
Finished <<< brat_robotic_description [0.64s]

Summary: 1 package finished [0.87s]

```

După aceasta compilare, trebuie să actualizăm sistemul.

```
$ source install/setup.bash
```

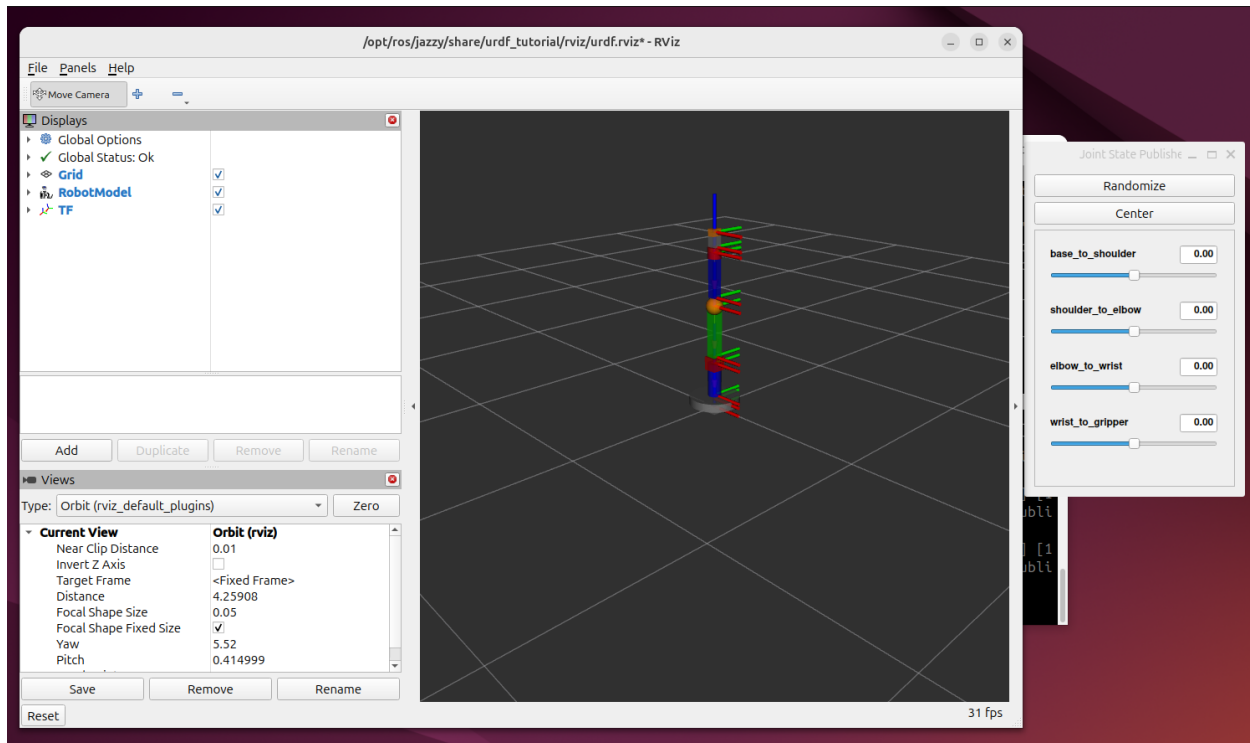
Pornim vizualizarea urdf-ului cu ajutorul comenzii (este scrisă pe o linie):

```

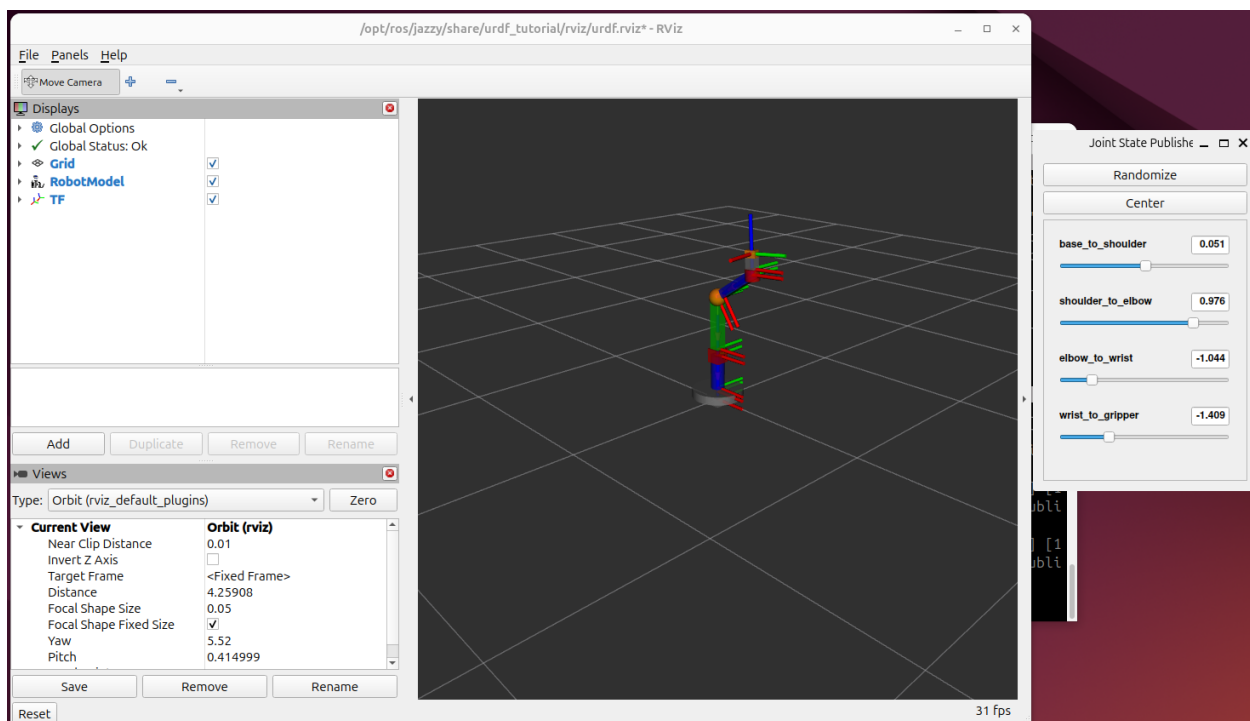
$ ros2 launch urdf_tutorial display.launch.py
model:=/home/thor/ros2_ws/src/brat_robotic_description/urdf/brat_robotic.urdf

```

Această comandă pornește pachetul pre-instalat urdf_tutorial cu care se face vizualizarea în ROS2 astfel, putem vizualiza robotul.



Cu ajutorul sliderelor se poate modifica pozitia fiecărei articulație.



3. Crearea pachetului de control și a nodului Python

Am creat un pachet de tip Python separat, dedicat controlului logic.

Navigam către fișierul src pentru a crea un pachet nou

```
$ cd ~/ros2_ws/src/
```

Am creat un pachet de tip Python care include dependențele necesare pentru comunicare (rclpy) și pentru mesajele senzorilor (sensor_msgs).

```
$ ros2 pkg create <nume_pachet> --build-type <tip_build> --dependencies <lista_dependente>
```

Elementele din comanda sunt următoarele:

- ros2 pkg create <nume_pachet> - comanda pentru a crea un pachet
- --build-type <tip_build> - se selectează tipul de sistem de build, pentru Python se folosește ament_python
- --dependencies <lista_dependențe> - adaugă una sau mai multe dependențe necesare pachetului

Acum putem genera automat structura pachetului cu ajutorul comenzii:

```
$ ros2 pkg create brat_robotic_control --build-type ament_python --dependencies rclpy  
sensor_msgs
```

Output

```
thor@thor-VM:~/ros2_ws/src$ ros2 pkg create brat_robotic_control --build-type am  
ent_python --dependencies rclpy sensor_msgs  
going to create a new package  
package name: brat_robotic_control  
destination directory: /home/thor/ros2_ws/src  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['thor <thor@todo.todo>']  
licenses: ['TODO: License declaration']  
build type: ament_python  
dependencies: ['rclpy', 'sensor_msgs']  
creating folder ./brat_robotic_control  
creating ./brat_robotic_control/package.xml  
creating source folder  
creating folder ./brat_robotic_control/brat_robotic_control  
creating ./brat_robotic_control/setup.py  
creating ./brat_robotic_control/setup.cfg  
creating folder ./brat_robotic_control/resource  
creating ./brat_robotic_control/resource/brat_robotic_control  
creating ./brat_robotic_control/brat_robotic_control/__init__.py  
creating folder ./brat_robotic_control/test  
creating ./brat_robotic_control/test/test_copyright.py  
creating ./brat_robotic_control/test/test_flake8.py  
creating ./brat_robotic_control/test/test_pep257.py  
  
[WARNING]: Unknown license 'TODO: License declaration'. This has been set in th  
e package.xml, but no LICENSE file has been created.  
It is recommended to use one of the ament license identifiers:  
Apache-2.0  
BSL-1.0  
BSD-2.0  
BSD-2-Clause  
BSD-3-Clause  
GPL-3.0-only  
LGPL-3.0-only  
MIT  
MIT-0
```

Navigăm în fișierul creat și folosim comanda ls pentru a verifica structura directorului.

```
$ cd brat_robotic_control
```

```
$ ls
```

Output

```
thor@thor-VM:~/ros2_ws/src/brat_robotic_control$ ls
brat_robotic_control package.xml resource setup.cfg setup.py test
```

Navigăm în fișierul brat_robotic_control și creem un fișier gol numit arm_controller.py

```
$ cd brat_robotic_control
```

```
$ touch arm_controller.py
```

Transformăm fișierul creat în executabil, astfel sistemul poate rula fișierul ca program.

```
$ chmod +x arm_controller.py
```

Mergem înapoi în rădăcina pachetului și deschidem Visual Studio Code.

```
$ cd ..
```

```
$ code .
```

După ce Visual Studio Code s-a deschis, căutăm și deschidem fișierul arm_control.py din directorul brat_robotic_control și scriem urmatorul cod:

```

brat_robotic_control > arm_controller.py > ArmController > __init__
1  #!/usr/bin/env python3
2
3  import rclpy
4  from rclpy.node import Node
5  from sensor_msgs.msg import JointState
6  import math
7
8  class ArmController(Node):
9      def __init__(self):
10         super().__init__('arm_controller')
11         # Cream publisher pe topicul /joint_states
12         self.publisher_ = self.create_publisher(JointState, 'joint_states', 10)
13
14         # Timer pentru publicare la fiecare 0.05 secunde (50ms)
15         self.timer = self.create_timer(0.05, self.publish_joint_states)
16
17         # Contorul de timp pentru miscari sinusoidale
18         self.time = 0.0
19
20         # Definim numele articulatiilor mobile
21         self.joint_names = [
22             'base_to_shoulder',
23             'shoulder_to_elbow',
24             'elbow_to_wrist',
25             'wrist_to_gripper'
26         ]
27
28         self.get_logger().info('Arm Controller a pornit - publică pe /joint_states')
29
30     def publish_joint_states(self):
31         # Cream mesajul JointState
32         msg = JointState()
33         msg.header.stamp = self.get_clock().now().to_msg()
34         msg.name = self.joint_names
35
36         # Calculam pozitii sinusoidale pentru fiecare articulatie
37
38         # Articulatia 1: Rotatie de 180 grade
39         pos1 = math.sin(self.time * 0.5) * 3.14
40
41         # Articulatia 2: Oscilatie la 90 de grade
42         pos2 = math.sin(self.time * 0.8) * 1.57
43
44         # Articulatia 3: Oscilatie cu faza diferita la 90 de grade
45         pos3 = math.sin(self.time * 0.6 + 1.0) * 1.57
46
47         # Articulatia 4: Rotatie de 180 grade
48         pos4 = math.sin(self.time * 1.2) * 3.14
49
50         msg.position = [pos1, pos2, pos3, pos4]
51
52         self.publisher_.publish(msg) # Publicam mesajul
53         self.time += 0.05 # Incrementam timpul
54
55         # Log periodic (la fiecare 2 secunde)
56         if int(self.time * 10) % 40 == 0:
57             self.get_logger().info(f'Poziții: [{pos1:.2f}, {pos2:.2f}, {pos3:.2f}, {pos4:.2f}]')
58
59     def main(args=None):
60         rclpy.init(args=args)
61         node = ArmController()
62         rclpy.spin(node)
63         node.destroy_node()
64         rclpy.shutdown()
65
66     if __name__ == '__main__':
67         main()

```

Tot în Visual Studio Code deschidem fișierul setup.py din directorul brat_robotic_control și adăugăm linia:

```
'arm_controller=brat_robotic_control.arm_controller:main'
```

După 'console_scripts': [

Astfel, transformăm scriptul Python într-o comandă executabilă pe care ROS2 o poate rula.

```
entry_points={
    'console_scripts': [
        'arm_controller=brat_robotic_control.arm_controller:main'
    ],
},
```

Acum vom compila pachetul din fișierul rădăcină al workspace-ului

```
$ cd ~/ros2_ws/
```

```
$ colcon build --packages-select brat_robotic_control
```

Output

```
thor@thor-VM:~/ros2_ws$ colcon build --packages-select brat_robotic_control
Starting >>> brat_robotic_control
Finished <<< brat_robotic_control [1.37s]

Summary: 1 package finished [1.53s]
```

După aceasta compilare, trebuie să actualizăm sistemul.

```
$ source install/setup.bash
```

4. Crearea fișierului launch

Vom crea un document demo_auto.launch.py în directorul launch din brat_robotic_description și vom deschide Visual Studio Code.

```
$ touch ~/ros2_ws/src/brat_robotic_description/launch/demo_auto.launch.py
```

```
$ cd ~/ros2_ws/src/brat_robotic_description/launch/
```

```
$ code .
```

Scriem în fișier următorul cod și vom compila pachetul din fișierul de rădăcină al workspace-ului:

```

launch > demo_auto.launch.py > generate_launch_description
1  from launch import LaunchDescription
2  from launch_ros.actions import Node
3  import os
4
5  def generate_launch_description():
6      # calea URDF
7      urdf_file = os.path.join(
8          os.path.dirname(__file__), '..', 'urdf', 'brat_robotic.urdf'
9      )
10
11     # citirea URDF
12     with open(urdf_file, 'r') as infp:
13         robot_desc = infp.read()
14
15     # calea RViz
16     rviz_config_file = os.path.join(
17         os.path.dirname(__file__), '..', 'rviz', 'brat_robotic.rviz'
18     )
19
20     return LaunchDescription([
21         # nod 1 Robot State Publisher
22         Node(
23             package='robot_state_publisher',
24             executable='robot_state_publisher',
25             parameters=[{'robot_description': robot_desc}]
26         ),
27
28         # nod 2 RViz pentru vizualizare
29         Node(
30             package='rviz2',
31             executable='rviz2',
32             arguments=['-d', rviz_config_file]
33         ),
34
35         # nod 3 controlerul python
36         Node(
37             package='brat_robotic_control', # numele pachetului de control
38             executable='arm_controller'    # numele executabilului de la setup.py
39         )
40     ])

```

\$ cd ~/ros2_ws/

\$ colcon build --packages-select brat_robotic_description

Output

```

thor@thor-VM:~/ros2_ws$ colcon build --packages-select brat_robotic_description
Starting >>> brat_robotic_description
Finished <<< brat_robotic_description [0.10s]

Summary: 1 package finished [0.26s]

```

După aceasta compilare, trebuie să actualizăm sistemul.

`$ source install/setup.bash`

Acum rulăm fișierul launch, care va deschide Rviz cu robotul care se mișcă la infinit, iar in terminal primim pozițiile articulațiilor.

`$ ros2 launch brat_robotic_description demo_auto.launch.py`

